



DMA (F411 techref, pag. 165)

9.1 DMA introduction

Direct memory access (DMA) is used in order to provide high-speed data transfer between peripherals and memory and between memory and memory. Data can be quickly moved by DMA without any CPU action. This keeps CPU resources free for other operations.

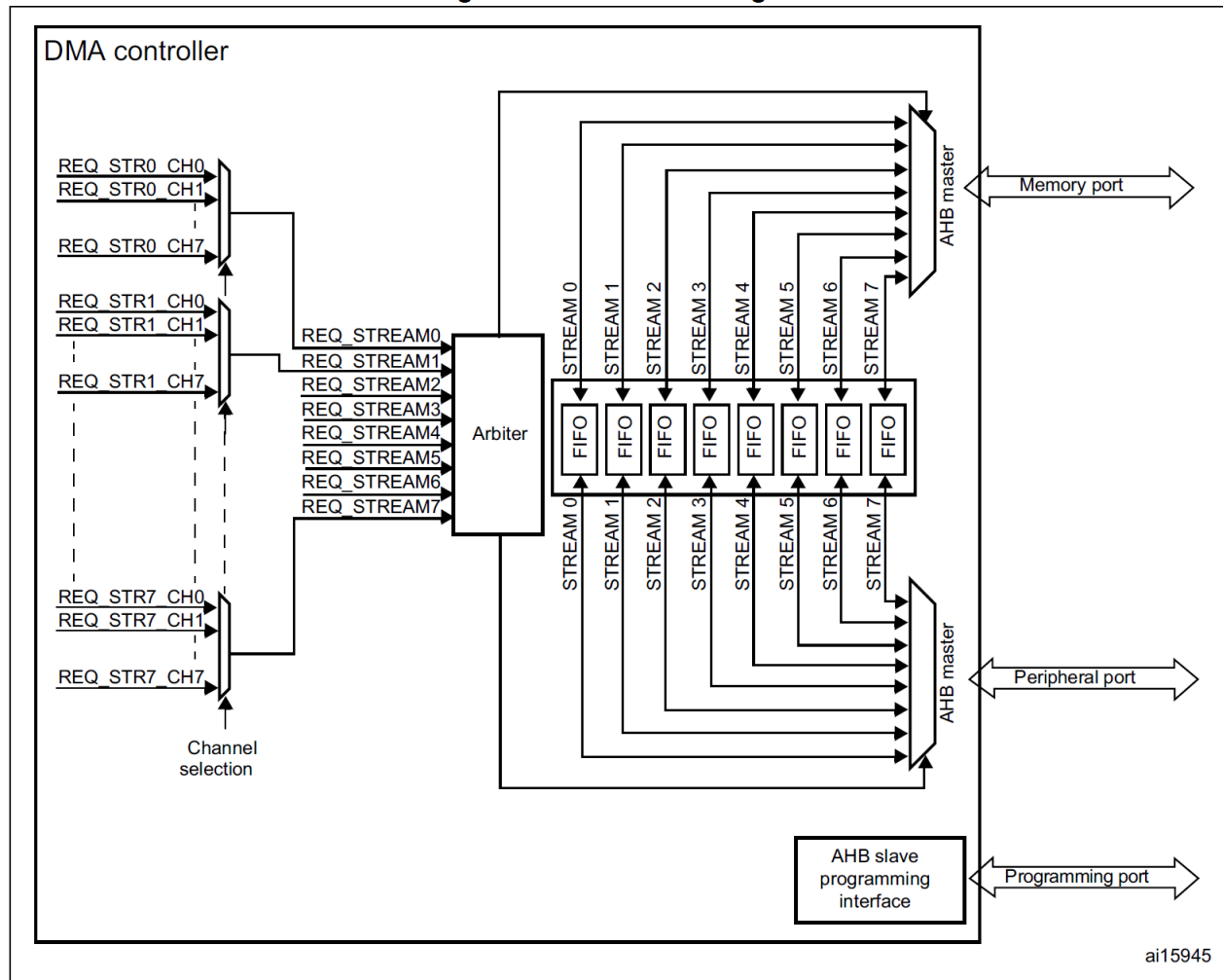
The DMA controller combines a powerful dual AHB master bus architecture with independent FIFO to optimize the bandwidth of the system, based on a complex bus matrix architecture.

The two DMA controllers have 16 streams in total (8 for each controller), each dedicated to managing memory access requests from one or more peripherals. Each stream can have up to 8 channels (requests) in total. And each has an arbiter for handling the priority between DMA requests.



DMA (F411 techref, pag. 167)

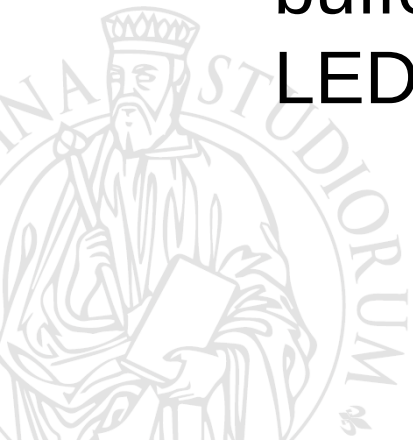
Figure 22. DMA block diagram





Lab Exercise:

- ADC and DMA:
 - Configure the ADC to sample a channel (PA0), triggered by a timer (TIM2)
 - Configure the DMA to store N samples in a circular buffer
 - In the DMA ISR, calculate the average of the buffer and put the output on a PWM on the LED (PA5, TIM2)





```
95  /* USER CODE BEGIN SysInit */
96
97  /* USER CODE END SysInit */
98
99  /* Initialize all configured peripherals */
100  MX_GPIO_Init();
101  MX_DMA_Init();
102  MX_USART2_UART_Init();
103  MX_TIM1_Init();
104  MX_ADC1_Init();
105  MX_TIM2_Init();
106  /* USER CODE BEGIN 2 */
107
108  uint16_t buff[20];
109  HAL_TIM_Base_Start(&htim1);
110  HAL_TIM_PWM_Start(&htim1,TIM_CHANNEL_1);
111  HAL_TIM_Base_Start(&htim2);
112  HAL_ADC_Start_DMA(&hadcl,(uint32_t *)buff,20);
113  /* USER CODE END 2 */
114
115  /* Infinite loop */
116  /* USER CODE BEGIN WHILE */
117  while (1)
118  {
119      /* USER CODE END WHILE */
120
121      /* USER CODE BEGIN 3 */
122  }
123  /* USER CODE END 3 */
```



```
201- /**
202   * @brief This function handles DMA2 stream0 global interrupt.
203   */
204- void DMA2_Stream0_IRQHandler(void)
205 {
206   /* USER CODE BEGIN DMA2_Stream0_IRQn 0 */
207   volatile int a;
208   if(__HAL_DMA_GET_IT_SOURCE(&hdma_adc1, DMA_IT_HT) != RESET)
209   {
210     a=0;
211   }
212   else
213   {
214     a=1;
215   }
216   /* USER CODE END DMA2_Stream0_IRQn 0 */
217   HAL_DMA_IRQHandler(&hdma_adc1);
218   /* USER CODE BEGIN DMA2_Stream0_IRQn 1 */
219
220   /* USER CODE END DMA2_Stream0_IRQn 1 */
221 }
222
223 /* USER CODE BEGIN 1 */
```